

Değerlendirme Metrikleri — Kapsamlı Açıklama

NIDS Projesi — VAE ve Dense Autoencoder Anomali Tespiti (Güncel Model, 19 Feature)

Tarih: 30 Temmuz 2026

Bu doküman, projede kullanılan her metriği üç düzeyde açıklıyor: **(1)** matematiksel tanım, **(2)** bu NIDS bağlamında ne anlama geldiği (hangi hatayı cezalandırıyor), **(3)** projenin kendi güncel (19 feature, concurrency_src_1s dahil) sonuçlarıyla somut bağlantı. Referans modeller: Dense autoencoder (`phase3\_dense/04\_phase3\_models/full\_features`, 5 seed) ve VAE (`phase3\_vae/05\_contamination\_sweep/04\_models/contam\_0pct`, 5 seed, deterministik z_mean skorum).

0. Temel Kavram: Confusion Matrix (Karışıklık Matrisi)

Bu proje, benign trafiği **negatif sınıf** (0), saldırı trafiğini **pozitif sınıf** (1) olarak alıyor. Bir modelin threshold_95 eşliğindeki her kararı 4 kutudan birine düşer:

	Gerçek: Benign	Gerçek: Saldırı
Model: Benign dedi	True Negative (TN) — doğru	False Negative (FN) — kaçırılan saldırı
Model: Saldırı dedi	False Positive (FP) — yanlış alarm	True Positive (TP) — doğru yakalanan saldırı

Dengesiz bir veri setinde (benign flow sayısı saldırı flow sayısından kat kat fazla) tek bir 'doğruluk (accuracy)' sayısı yanıltıcıdır — her şeyi 'benign' diyen bir model bile yüksek accuracy alabilir. Bu yüzden proje boyunca **hangi hatanın** (FN mi, FP mi) ölçüldüğünü açıkça ayıran metrikler kullanılıyor.

1. Recall (Attack Recall / Duyarlılık)

Tanım: $Recall = TP / (TP + FN)$ — gerçek saldırı flow'larının kaçta kaçının yakalandığı.

NIDS bağlamında anlamı: Bir güvenlik ekibi için en kritik metriklerden biri — **kaçırılan saldırı** oranını doğrudan gösterir. Recall düşükse, model o saldırı tipini büyük ölçüde görmezden geliyor demektir; bu proje boyunca 'zayıf tespit' derken kastedilen budur.

Projedeki somut örnek: apache_bench'in recall'u, eski 18-feature modelde sadece %2,6-3,3 iken, concurrency_src_1s eklenince Dense'te %90,9'a, VAE'de %95,0'e çıktı — bu raporun en önemli tek sayısı budur (bkz. Proje_Sonucular_TR.pdf, Bölüm 5).

2. ROC-AUC

Tanım: ROC eğrisi, olası her eşik değeri için True Positive Rate'i (=recall) False Positive Rate'e karşı çizer. AUC (Area Under Curve), bu eğrinin altındaki alan — 0,5 = rastgele tahmin, 1,0 = kusursuz ayırım.

NIDS bağlamında anlamı: Eşikten bağımsız bir ayırım gücü ölçüsü — 'model, saldırı flow'larına benign flow'lardan sistematik olarak daha yüksek bir anomali skoru veriyor mu?' sorusuna cevap verir, belirli bir threshold_95 seçimine bağlı değildir. Bu yüzden projede hem ROC-AUC hem de eşiğe bağlı recall/F1/FPR birlikte raporlanıyor: biri modelin ham ayırım gücünü, diğeri gerçek çalışma noktasındaki davranışını gösteriyor.

Projedeki somut örnek: apache_bench ROC-AUC'u eski modelde 0,58-0,70 (rastgele tahminin biraz üzerinde) iken, yeni modelde 0,98'in üzerine çıktı — modelin apache_bench'i benign'den ayırma gücü artık neredeyse kusursuz.

3. PR-AUC (Precision-Recall Alan)

Tanım: Precision = TP / (TP + FP) — modelin 'saldırı' dediği flow'ların kaçta kaçının gerçekten saldırı olduğu. PR eğrisi, precision'ı recall'a karşı çizer; PR-AUC bu eğrinin altındaki alandır (= Average Precision, AP).

NIDS bağlamında anlamı: ROC-AUC'un aksine, PR-AUC sınıf dengesizliğine duyarlıdır — saldırı flow'ları benign'e göre çok azınlıktaysa (bu projede apache_bench değerlendirmesinde saldırı prevalansı ~%17,9), PR-AUC yanlış alarmların pratikteki maliyetini ROC-AUC'tan daha iyi yansıtır. Baseline (rastgele model) PR-AUC'u, prevalansa eşittir — bu yüzden tablolarda 'baseline (prevalence = ...)' çizgisiyle birlikte gösteriliyor.

Projedeki somut örnek: apache_bench PR-AUC'u Dense'te 0,893, VAE'de 0,904-0,932 — prevalansın (0,179) çok üzerinde, yani model 'saldırı' dediğinde büyük çoğunlukla haklı.

Not: Bu projedeki PR-AUC/F1 sayıları, O3 dedup-prevalence düzeltmesi (Bölüm 6, Proje_Sonucular_TR.pdf) güncel modele henüz yeniden uygulanmadığından *geçici* kabul edilmeli — resampled pencerelerin bazı flow'ları test setinde iki kez sayması, prevalansı (ve dolayısıyla PR-AUC/F1'i) hafifçe etkileyebilir. ROC-AUC/recall/FPR bundan etkilenmez.

4. F1 Score

Tanım: $F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$ — precision ve recall'un harmonik ortalaması, belirli bir eşikte (bu projede threshold_95) hesaplanır.

NIDS bağlamında anlamı: Tek bir sayıda 'hem kaçırma hem yanlış alarm' dengesini özetler — ama **hangi hatanın daha maliyetli olduğunu ayırt etmez** (bir güvenlik ekibi genelde kaçırmayı yanlış alarmdan çok daha pahalı bulur, F1 bunu yansıtmaz). Bu yüzden projede F1 tek başına değil, recall/FPR/ROC-AUC ile birlikte okunuyor.

Projedeki somut örnek: apache_bench F1'i 0,040-0,051'den 0,82-0,84'e çıktı — recall'daki büyük sıçramanın F1'e yansması.

5. Benign FPR (False Positive Rate)

Tanım: $FPR = FP / (FP + TN)$ — benign flow'ların kaçta kaçının yanlışlıkla saldırı diye işaretlendiği.

NIDS bağlamında anlamı: Bir güvenlik ekibinin katlanmak zorunda kalacağı 'yalancı alarm' yükü. threshold_95 tanımı gereği (benign validasyon setinin %95. persentili), FPR yapısal olarak ~%5 civarında olması **beklenir** — gözlenen sapmalar (bu projede %5,9-6,6 arası), val→test transferindeki küçük sistematik sapmayı ve concurrency_src_1s'in benign

yüksek-frekans trafiğe kısmi etkisini yansıtıyor.

Projedeki somut örnek: concurrency_src_1s eklenmeden önce benign FPR ~%6,15 (Dense) / %5,77 (VAE) idi; sonra ~%6,60 / %6,64'e çıktı — apache_bench recall'undaki devasa kazanca kıyasla küçük bir maliyet (+0,4-0,5 puan).

6. threshold_95

Tanım: Held-out (eğitime hiç girmemiş) benign doğrulama flow'larının reconstruction-error dağılımının **95. persentili**. Bir flow'un hatası bu değeri aşarsa 'saldırı' diye işaretlenir.

NIDS bağlamında anlamı: 'Benign trafiğin en kötü %5'i kadar anormal görünen her şeyi alarma çevir' kuralı — tasarım gereği FPR'ı yaklaşık %5'e sabitler. Bu değer her seed için **ayrı ayrı yeniden hesaplanır** (seed'ler arası model varyansını val setine göre telafi eder) ve VAE için deterministik z_mean skoru üzerinden, Dense için kendi reconstruction error'u üzerinden hesaplanır — iki model kendi hata dağılımına göre kalibre edilir, ortak bir sabit sayı değildir.

Bilinen sınırlama: Küçük bir val setinden (VAE için 653 flow) tahmin edilen bir kuyruk persentili gürültülü olabilir; ayrıca val→test transferinde küçük bir sistematik yukarı sapma gözlenmiştir (gerçekleşen FPR nominal %5'in biraz üzerinde) — detay için final rapordaki 'Threshold calibration note'a bakınız.

7. KS İstatistiği (Kolmogorov-Smirnov Testi)

Tanım: İki dağılımın (örn. bir feature'ın benign'deki değerleri ile aynı feature'ın apache_bench'teki değerleri) ampirik kümülatif dağılım fonksiyonları (CDF) arasındaki **en büyük dikey mesafe**. 0 = dağılımlar özdeş, 1 = tamamen ayrık.

NIDS bağlamında anlamı: Retrain yapmadan, ucuza 'bu feature saldırı tipini ayırt etmede işe yarar mı?' sorusunu ön-test etmek için kullanıldı — hem eski 18 feature'ın apache_bench için neden yetersiz kaldığını teşhis etmekte (Bölüm 4.1, Proje_Sonucular_TR.pdf), hem de yeni feature adaylarını (IAT, concurrency) retrain'e sokmadan önce elemekte/onaylamakta kullanıldı.

Kritik uyarı — projenin kendi bulgusu: Yüksek KS, tek başına 'bu feature işe yarayacak' anlamına gelmez. apache_bench'in en iyi eski feature'ları KS=0,62-0,76 veriyordu ama işe yaramıyordu — çünkü apache_bench'in ortalaması benign'in normal aralığının **içindeydi** (sadece 0,4-0,7 standart sapma uzakta), yüksek KS sadece apache_bench'in çok dar/düşük-varyanslı bir küme olmasından kaynaklanıyordu. Buna karşılık concurrency_src_1s'in KS'i benzer aralıktaydı (0,86-0,99) ama ortalama kayması çok daha büyüktü (3-4 standart sapma) — retrain'de işe yarayan da bu oldu. Kısacası: **KS'e ek olarak ortalama-kayma büyüklüğüne (effect size) bakmadan bir feature hakkında karar vermeyin** — bu proje bunun hem başarısız (IAT) hem başarılı (concurrency) örneğini içeriyor.

8. Reconstruction Error (Yeniden İnşa Hatası)

Tanım: Bir autoencoder, girdisini sıkıştırıp (bottleneck) tekrar açar; reconstruction error, girdi ile bu yeniden inşa edilmiş çıktı arasındaki ortalama kare farkıdır (MSE): $error = mean((x - \hat{x})^2)$.

NIDS bağlamında anlamı: Model sadece benign trafikle eğitildiğinden, 'normal'in nasıl sıkıştırılıp açılacağını öğrenir. Benign bir flow verildiğinde düşük hatayla yeniden inşa edilir; modelin daha önce görmediği bir kalıp (saldırı) verildiğinde ise yeniden inşa daha kötü olur — **anomali skoru budur**. `threshold_95` bu skor üzerinde tanımlanır.

VAE'ye özel not — deterministik z_mean skora: VAE'nin bottleneck'i olasılıksal (`z_mean`, `z_log_var`); orijinal skora yöntemi her hesaplamada `z`'yi rastgele örnekliyordu (reparametrizasyon trick'i), bu da aynı flow'un skorunun çalıştırmadan çalıştırmaya değişmesine yol açıyordu (bağımsız denetim bulgusu O2). Bu proje, `z = z_mean` kullanarak (örnekleme yapmadan) **deterministik** bir skora geçti — `concurrency_src_1s` ile yapılan VAE retrain'i (VAE v2) baştan bu yöntemle skorlanmıştır.

9. Pooled Recall vs. Decomposed Recall

Tanım: İkili (pairwise) değerlendirmede, aynı test setinde iki farklı saldırı tipi birlikte bulunur. **Pooled recall**, bu karışık settteki TÜM saldırı flow'larının (iki tipin toplamı) ortalama recall'udur. **Decomposed recall** ise aynı setin içinden sadece belirli bir tipin (örn. sadece `apache_bench`) flow'larını ayırıp onun kendi recall'unu hesaplar.

NIDS bağlamında anlamı — neden ikisi de gerekli: Pooled recall yanıltıcı olabilir: iyi tespit edilen bir tip (`portscan`, `recall≈1,0`) karışıma eklendiğinde, pooled recall **mekanik olarak** yükselir — `apache_bench`'in kendi tespitinde hiçbir gerçek iyileşme olmasa bile. Decomposed recall bu karışım etkisini ayıklar ve 'model gerçekten bu tipi daha mı iyi tespit ediyor?' sorusuna dürüst cevap verir.

Projedeki somut örnek: `portscan` + `apache_bench` çiftinde pooled recall %93,8 (Dense) / %96,6 (VAE) görünüyor — ama `apache_bench`'in kendi (decomposed) recall'u tam olarak solo değerlendirmedeki gibi kalıyor (Dense: %90,9; VAE: %95,0), hangi tipte eşleştirildiğinden bağımsız. Bu, modelin her flow için verdiği kararın statik olduğunu (eşit eşik, flow-bazlı, komşu flow'lardan etkilenmeyen) somut olarak doğruluyor.

10. concurrency_src_1s — Yeni Feature'ın Mantığı

Tanım: Bir flow için, o flow'un kendi zaman damgası (`ts`) etrafında ± 1 saniyelik pencerede, **aynı kaynak IP'den** gelen başka kaç flow olduğunun sayısı. Formül olarak: aynı `window_id` içinde, aynı `id.orig_h`'ye sahip flow'lar zaman damgasına göre sıralanır, her flow için $|\Delta t| \leq 1s$ olan komşularının sayısı (kendisi hariç) sayılır.

Neden bu şekilde tasarlandı:

- **Log1p dönüşümü** — ham sayım feature'ı sağa çarpık (çoğu flow'un yakınında az sayıda komşu var, birkaçının çok fazla); `log1p` bu çarpıklığı azaltıp modelin öğrenmesini kolaylaştırıyor.
- **Scaler sadece benign-train split'ine fit edildi** — projenin her feature'da uyguladığı leakage-free kuralın aynısı; val/test bilgisi ölçeklemeye hiç karışmadı.
- **Hiçbir yerde sabit bir IP değeri yok** — hesaplama tamamen veri-güdümlü (o flow'un gerçek kaynak IP'si neyse ona göre gruplanıyor); bu, modelin belirli bir IP adresini ezberlemesini değil, genel bir davranış kalıbını (yüksek istek hızı) öğrenmesini sağlamak için kasıtlı bir tasarım kararı.

- **Neden ± 1 saniye:** Denenen 3 yarıçaptan ($\pm 1s/\pm 2s/\pm 5s$) apache_bench için en güçlü ve en yorumlanabilir ayrımı $\pm 1s$ verdi (bkz. 14_concurrency_feature_experiment/), ve knock-out testinde (feature dondurulup model aynen değerlendirildiğinde) iyileşmenin gerçekten bu feature'dan geldiği doğrulandı.

Neden işe yaradı: apache_bench'i benign HTTP trafiğinden ayıran şey, tek bir isteğin şekli değil — aynı isteğin çok kısa aralıklarla defalarca tekrarlanması. Eski 18 feature'ın hepsi tek bir flow'u kendi başına değerlendiriyordu; concurrency_src_1s, ilk kez flow'lar **arasındaki** ilişkiyi (yerel istek yoğunluğu) modele veren feature oldu.

Genellenebilirlik uyarısı: Bu laboratuvar veri setinde tek bir saldırgan IP var; feature IP-agnostik tasarlanmış olsa da, veri setinin kendisi 'yüksek istek hızı = şüpheli' ile 'yüksek istek hızı = bu spesifik saldırgan' ayrımını yapamaz. Çok-kaynaklı (NAT, load balancer, meşru yüksek-hızlı istemciler) bir ortamda bu feature'ın benign false-positive davranışı doğrulanmadan production'a taşınmamalı (bkz. Proje_Sonuclar_TR.pdf, Bölüm 4.3).

Özet Tablo — Hangi Metriğe Ne Zaman Bakmalı

Soru	Bakılacak metrik
Model genel olarak bu saldırı tipini ayırt edebiliyor mu (eşikten bağımsız)?	ROC-AUC
Dengesiz sette (az saldırı, çok benign) ayırım gücü ne kadar?	PR-AUC
Gerçek çalışma noktasında (threshold_95) kaç saldırı kaçırılıyor?	Recall
Gerçek çalışma noktasında kaç yanlış alarm var?	Benign FPR
Recall ve precision'ı tek sayıda özetle (eşitlik varsayımıyla)	F1
Bu feature retrain'e değer mi (ucuz ön-test)?	KS + ortalama-kayma büyüklüğü
Bir çiftteki iyileşme gerçek mi, karışım etkisi mi?	Decomposed vs. pooled recall